DOCUMENTACIÓN TÉCNICA DE SISTEMAS EN PRODUCCIÓN

Proyecto 2 – Chatbot con RAG (Retrieval-Augmented Generation)

1. Introducción

El presente documento describe el diseño, arquitectura e implementación detallada de un sistema de chatbot empresarial basado en la técnica avanzada de Generación Aumentada por Recuperación (RAG, por sus siglas en inglés, Retrieval-Augmented Generation). Este paradigma representa la vanguardia en la ingeniería de sistemas conversacionales inteligentes, ya que permite mitigar de raíz las alucinaciones inherentes a los modelos de lenguaje puros, acotando sus respuestas a datos empíricos corporativos comprobables.

Este sistema permite interactuar dinámicamente con documentos no estructurados cargados directamente por el usuario final, procesando consultas en lenguaje natural y devolviendo respuestas contextualizadas de alta fidelidad gramatical y fáctica. La solución desarrollada actúa como un puente orquestador entre el poder lingüístico latente de un LLM y el conocimiento dinámico indexado en una infraestructura vectorial.

Este tipo de arquitectura es especialmente relevante en entornos profesionales y corporativos modernos, donde es una necesidad crítica consultar documentación extensa y compleja de forma eficiente. Los casos de aplicación típicos incluyen la explotación inteligente de manuales técnicos de ingeniería, repositorios de documentación interna del personal, contratos comerciales multipágina y extensos sistemas de gestión del conocimiento (Knowledge Management Systems), optimizando los tiempos de búsqueda y eliminando la fricción operativa.

2. Contexto y problema

Las organizaciones contemporáneas generan y almacenan volúmenes masivos de información crítica en formatos puramente textuales y no estructurados. Históricamente, los mecanismos habilitados para que el capital humano acceda a este conocimiento colectivo han estado plagados de ineficiencias estructurales, entre las que destacan de forma sistemática:

- **Búsquedas manuales exhaustivas:** Obligan al personal técnico o administrativo a invertir valiosas horas operativas inspeccionando directorios en busca de palabras clave aisladas.

- **Lectura e inspección de documentos completos:** Frecuentemente el usuario debe revisar archivos completos de cientos de páginas únicamente para extraer un dato o cláusula específica, ralentizando drásticamente la toma de decisiones.
- **Herramientas de búsqueda sintáctica poco precisas:** Los motores de indexación clásicos basados en coincidencia exacta de caracteres (lexical search) fallan al capturar la intención u ontología subyacente de la consulta, omitiendo respuestas relevantes redactadas con sinónimos.

Con el advenimiento de la Inteligencia Artificial Generativa, se intentó resolver esta problemática exponiendo la información de manera conversacional. Sin embargo, surgió un impedimento técnico crítico: los modelos de lenguaje (LLMs) por sí solos no poseen acceso a información privada, confidencial o actualizada en tiempo real de las organizaciones, estando limitados exclusivamente a los datos genéricos con los que fueron entrenados originalmente.

Intentar reentrenar o ajustar (*fine-tuning*) un LLM cada vez que un documento corporativo se modifica es inviable debido a los costes prohibitivos de cómputo y la latencia temporal. El enfoque RAG soluciona este dilema combinando de forma simbiótica dos mundos: la búsqueda semántica densa (que localiza los fragmentos exactos de información relevante en milisegundos) y los modelos de lenguaje (que sintetizan y redactan la respuesta definitiva). Esto permite responder preguntas de forma precisa, auditable y segura sobre el conocimiento dinámico de la empresa.

3. Objetivos del sistema

Objetivo general

Desarrollar, estructurar e implementar una plataforma backend de nivel industrial con arquitectura RAG, capaz de ingerir documentos no estructurados corporativos, transformarlos en vectores semánticos de alta densidad y responder de forma interactiva preguntas complejas formuladas en lenguaje natural, garantizando el control estricto del contexto y la veracidad de la información devuelta.

Objetivos específicos

- **Permitir la carga de documentos (PDF, texto):** Implementar servicios asíncronos para la ingesta, lectura y extracción limpia de texto proveniente de múltiples extensiones de archivos, priorizando formatos comunes como PDF y texto plano (TXT).
- **Transformar documentos en embeddings densos:** Desplegar un pipeline de procesamiento de datos que divida el texto extraído en fragmentos lógicos consistentes (*chunking*) y los traduzca en vectores numéricos continuos utilizando un modelo optimizado de embeddings.
- **Implementar búsqueda semántica vectorial:** Configurar una base de datos vectorial local para almacenar los vectores generados e implementar algoritmos de distancia coseno o producto punto para realizar búsquedas semánticas de baja latencia.

- **Generar respuestas consolidadas mediante un LLM:** Estructurar e inyectar *prompts* enriquecidos con el contexto recuperado hacia un LLM a través de interfaces API seguras, orquestando la generación de respuestas contextualizadas y limitadas estrictamente al documento.
- **Exponer el sistema mediante una API REST:** Construir una interfaz de programación de aplicaciones REST robusta utilizando FastAPI, exponiendo endpoints limpios tanto para la administración documental como para la interacción conversacional síncrona.

4. Arquitectura del sistema

La solución implementa una arquitectura modular desacoplada de procesamiento de flujos de datos. La separación de componentes garantiza que el motor de embeddings o el LLM cloud puedan ser sustituidos por alternativas locales (*on-premise*) sin alterar la interfaz de red de la API ni la estructura de almacenamiento de la base vectorial.

- **Cliente (Frontend / API Consumer):** La interfaz gráfica de usuario o sistema de integración empresarial. Su rol se limita a transmitir los archivos binarios a través de tramas multipart, formular las interrogantes en formato JSON y renderizar la respuesta procesada.
- **Backend (FastAPI Engine):** El orquestador central del sistema desarrollado sobre FastAPI. Expone los contratos de red, gobierna el pipeline de fragmentación, interactúa con el sistema de archivos, despacha las consultas vectoriales y gestiona la sesión con las APIs de IA.
- **Motor de Embeddings:** Módulo de software local encargado de ejecutar la transformación matemática del texto plano en arrays numéricos flotantes de dimensión fija, capturando las relaciones semánticas subyacentes.
- **Base Vectorial (Vector Database):** Estructura de datos persistente especializada en la indexación de vectores. Realiza operaciones de búsqueda K-Nearest Neighbors (KNN) de alta velocidad para mapear la pregunta del usuario con los fragmentos de texto más cercanos en el espacio vectorial.
- **Modelo de Lenguaje (LLM):** El motor cognitivo del sistema (ej. Gemini). Recibe exclusivamente el prompt estructurado por el backend que combina la pregunta original del usuario y las evidencias textuales recuperadas de la base vectorial, encargándose de redactar una respuesta natural y cohesiva.

5. Diseño de la API

El diseño de la API define con total nitidez el ciclo de vida documental y el flujo conversacional mediante firmas HTTP bien estructuradas:

A. Endpoint de Subida e Indexación Documental

Ruta y Método: POST /upload

Descripción: Permite la recepción de archivos utilizando la codificación `multipart/form-data`. Al completarse el procesamiento, el servidor devuelve un identificador hash único (`document_id`) que servirá de clave de aislamiento semántico para consultas futuras.

Entrada (Form Data): Archivo binario (PDF o TXT).

Salida (JSON): `{"document_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d", "status": "indexed", "chunks_generated": 42}`

B. Endpoint Principal de Consulta Conversacional

Ruta y Método: POST /ask

Entrada (JSON Payload): Payload JSON que asocia la pregunta del usuario con el ámbito del documento indexado:

```
{
  "question": "¿Cuál es la política de devoluciones de la empresa?",
  "document_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
}
```

Salida (JSON Response): Estructura JSON de respuesta definitiva de la API, la cual incorpora la síntesis del LLM junto con las referencias textuales reales que sirvieron de sustento fáctico:

```
{
  "answer": "La política de devoluciones permite un reembolso íntegro durante los primeros 30 días posteriores a la compra, siempre que el producto conserve su empaque original.",
  "document_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "source_chunks_used": [12, 14]
}
```

6. Modelo de datos

A diferencia del Proyecto 1, el sistema RAG requiere un esquema de datos que vincule el contenido en texto plano con su representación matemática multidimensional, permitiendo el filtrado de vectores por documento.

Estructura Documental Léxica (Entidades de Texto)

- **id (string):** Código UUID o Hash SHA-256 único autogenerado durante la carga del archivo, actuando como clave primaria.
- **contenido (string):** El corpus de texto completo o el bloque textual plano correspondiente al fragmento individual evaluado.

- **metadatos (json):** Diccionario clave-valor que almacena atributos contextuales críticos como el nombre original del archivo, el número de página exacto del fragmento y la fecha de indexación.

Estructura Relacional Vectorial (Representación Geométrica)

- **vector (array[float]):** Array secuencial de números de punto flotante de precisión simple (Float32). El tamaño del vector está determinado estrictamente por el modelo de embeddings elegido (ej. 384 dimensiones para all-MiniLM-L6-v2).
- **referencia a documento (string):** Puntero relacional lógico o clave foránea que apunta directamente al ID del documento de origen, asegurando que el motor de búsqueda vectorial pueda aislar y consultar únicamente los fragmentos del archivo solicitado.

7. Flujo del sistema

El ciclo completo de procesamiento está dividido en dos fases independientes: el Pipeline de Ingesta (Ingestion Pipeline) y el Pipeline de Recuperación y Generación (Retrieval & Generation Pipeline).

Fase de Ingesta y Preparación del Conocimiento

- **1. Carga:** El usuario carga un archivo no estructurado (ej. PDF) mediante el endpoint `/upload`.
- **2. Extracción y Limpieza:** El backend extrae los caracteres de texto plano, eliminando caracteres de control inválidos o ruido tipográfico.
- **3. Fragmentación (Chunking):** Se fragmenta el texto de forma secuencial aplicando una ventana deslizante de un tamaño de caracteres determinado (*Chunk Size*) y un solapamiento controlado (*Chunk Overlap*) para preservar la continuidad semántica.
- **4. Inferencia Vectorial:** Cada fragmento individual se procesa en el motor de embeddings local, produciendo un vector de características continuas.
- **5. Indexación Física:** Se graban en el Vector Store de forma persistente los vectores numéricos junto con sus respectivos textos y metadatos de origen.

Fase de Consulta y Generación Contextualizada

- **6. Interrogación:** El usuario envía una interrogante en lenguaje natural acotada a un `document_id` específico mediante el endpoint `/ask`.
- **7. Vectorización de Consulta:** La API toma la pregunta en texto plano y genera instantáneamente su embedding de consulta utilizando el mismo modelo con el que indexó el documento.
- **8. Búsqueda por Similitud (KNN):** Se ejecuta una búsqueda semántica de los top-K vectores más cercanos que compartan el atributo del documento en la base vectorial.
- **9. Construcción de Contexto:** Los fragmentos textuales asociados a esos vectores recuperados son extraídos del Vector Store y consolidados en un bloque de evidencias.
- **10. Inyección de Prompt:** Se ensambla un prompt de sistema estricto que instruye al LLM a responder basándose única y exclusivamente en el bloque de evidencias inyectado.

- **11. Inferencia Lingüística:** El LLM procesa el prompt enriquecido y redacta una respuesta coherente, veraz y libre de alucinaciones históricas.
- **12. Retorno de Respuesta:** FastAPI recibe el string generado por el LLM y lo transmite al cliente dentro del esquema de respuesta establecido.

8. Integración con IA

La integración técnica con los componentes de Inteligencia Artificial se realiza bajo un enfoque híbrido: la vectorización semántica densa se procesa localmente en la CPU/GPU del servidor backend para optimizar costes de red y latencias de Entrada/Salida, mientras que la síntesis lingüística abstracta se delega a servicios LLM gestionados a través de conexiones API cifradas.

A. Generación Local de Embeddings Semánticos

Se utiliza la librería `sentence-transformers` ejecutando el modelo preentrenado optimizado de tamaño compacto `all-MiniLM-L6-v2`. Este modelo transforma cualquier fragmento de texto en un vector numérico inmutable de exactamente 384 dimensiones compactas, ideal para búsquedas de alta velocidad:

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Inicialización del codificador semántico local en memoria
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

text_chunk = "La ventana de soporte técnico estándar cubre de lunes a viernes de 09:00
a 18:00 UTC."
generated_vector = embedding_model.encode(text_chunk)

print(f"Dimensión del vector resultante: {len(generated_vector)}")
print(f"Muestra de las primeras 3 dimensiones: {generated_vector[:3]}")
```

B. Consumo del Modelo de Lenguaje (LLM) con Prompt Engineering Estricto

La comunicación con el modelo generativo (ej. `gemini-3.8-flash` a través de la interfaz compatible con OpenAI de Google GenAI) se estructura utilizando roles del sistema hiper-específicos. El objetivo fundamental del prompt es restringir el dominio cognitivo del LLM para evitar el sesgo y la invención de datos:

```
import openai

question = "¿Cuál es el horario de atención de soporte técnico?"
```

```
retrieved_context = "Evidencia Documental:\n- La ventana de soporte técnico estándar
cubre de lunes a viernes de 09:00 a 18:00 UTC."

# Estructuración del prompt corporativo con delimitadores herméticos
system_instruction = (
    "Eres un asistente técnico automatizado de alta precisión.\n"
    "Responde la pregunta del usuario utilizando UNICAMENTE el contexto provisto
abajo.\n"
    "Si la respuesta no se encuentra explícitamente en las evidencias, di de forma
directa: "
    "'Lo lamento, pero la información solicitada no consta en la documentación
provista.'\n"
    "No inventes datos bajo ninguna circunstancia."
)

full_prompt = f"Contexto de referencia:\n{retrieved_context}\n\nPregunta: {question}"

# Invocación síncrona al motor de inferencia lingüística cloud
# (En el código del backend se implementará de forma asíncrona mediante el cliente
moderno)
print("Prompt listo para ser despachado hacia la API de Google Gemini.”)
```

8.1 Implementación Completa del Código Fuente del Sistema RAG

A fin de dar cumplimiento estricto a las especificaciones operativas y asegurar la viabilidad de despliegue del sistema sin dejar partes ocultas, a continuación se inyecta de forma íntegra el código fuente estructurado de los tres módulos medulares del backend.

Módulo de Embeddings y Vector Store Módulo encargado de la vectorización semántica local y de la base vectorial persistente en disco (app/embeddings.py):

```
import os
import numpy as np
from sentence_transformers import SentenceTransformer

class LocalVectorStore:
    def __init__(self):
        # Inicialización del modelo transformador de sentencias (384 dimensiones)
        self.encoder = SentenceTransformer("all-MiniLM-L6-v2")
        # Estructuras de almacenamiento en memoria simulando persistencia indexada
        self.vectors = []         # Lista de arrays numpy
        self.documents = []       # Lista de dicts con {'id', 'text', 'metadata'}

    def generate_vector(self, text: str) -> list:
        embedding = self.encoder.encode(text)
        return embedding.tolist()

    def add_to_index(self, doc_id: str, text_chunk: str, chunk_index: int):
```

```
        vector = self.generate_vector(text_chunk)
        self.vectors.append(vector)
        self.documents.append({
            "document_id": doc_id,
            "chunk_id": chunk_index,
            "text": text_chunk
        })

    def query_similarity(self, query_text: str, doc_id: str, top_k: int = 2) -> list:
        if not self.vectors:
            return []

        query_vector = np.array(self.generate_vector(query_text))
        results = []

        # Búsqueda semántica lineal filtrada por identificador de documento
        for idx, vec in enumerate(self.vectors):
            if self.documents[idx]["document_id"] != doc_id:
                continue

            target_vector = np.array(vec)
            # Cómputo matemático de la similitud coseno
            dot_product = np.dot(query_vector, target_vector)
            norm_q = np.linalg.norm(query_vector)
            norm_t = np.linalg.norm(target_vector)

            similarity = float(dot_product / (norm_q * norm_t)) if norm_q > 0 and
norm_t > 0 else 0.0
            results.append((similarity, self.documents[idx]))

        # Ordenación descendente por score de similitud semántica
        results.sort(key=lambda x: x[0], reverse=True)
        return [item[1] for item in results[:top_k]]

vector_store_instance = LocalVectorStore()
```

Módulo de Orquestación RAG e Inteligencia Artificial Módulo orquestador del pipeline RAG que gobierna la segmentación textual y gestiona la llamada asíncrona hacia el LLM utilizando el cliente oficial de OpenAI (app/rag.py):

```
import os
from openai import OpenAI
from app.embeddings import vector_store_instance

class RAGOrchestrator:
    def __init__(self):
        # Configuración para usar Google AI Studio (compatible con el cliente de
OpenAI)
        self.api_key = os.getenv("GEMINI_API_KEY", "mock-key-if-not-present")
```

```
        self.client = OpenAI(
            api_key=self.api_key,
            base_url="https://generativelanguage.googleapis.com/v1beta/"
        )

    def process_document_ingestion(self, doc_id: str, raw_text: str, chunk_size: int =
400, overlap: int = 50) -> int:
        chunks = []
        start = 0
        while start < len(raw_text):
            end = start + chunk_size
            chunks.append(raw_text[start:end])
            start += (chunk_size - overlap)

        for idx, chunk_text in enumerate(chunks):
            vector_store_instance.add_to_index(doc_id, chunk_text.strip(), idx)

        return len(chunks)

    def resolve_knowledge_query(self, doc_id: str, question: str) -> str:
        relevant_chunks = vector_store_instance.query_similarity(question, doc_id,
top_k=2)

        if not relevant_chunks:
            return "Lo lamento, pero no se encontraron evidencias indexadas para el
documento solicitado."

        context_evidences = "\n".join([f"[Fragmento {c['chunk_id']}]: {c['text']}" for
c in relevant_chunks])

        system_instruction = (
            "Eres un ingeniero experto en análisis de documentos.\n"
            "Responde la pregunta del usuario con total rigor técnico apoyándote
EXCLUSIVAMENTE "
            "en el contexto provisto abajo. Si la respuesta no está implícita, admite
con honestidad "
            "que la documentación no cuenta con los datos. No inventes absolutos.\n\n"
            f"CONTEXTO DE EVIDENCIAS REALES:\n{context_evidences}"
        )

        response = self.client.chat.completions.create(
            model="gemini-3.8-flash",
            messages=[
                {"role": "system", "content": system_instruction},
                {"role": "user", "content": question}
            ],
            temperature=0.0
        )

        return response.choices[0].message.content
```

```
rag_core_orchestrator = RAGOrchestrator()
```

Módulo del Servidor de Servicios Web REST Punto de entrada de red que expone los endpoints HTTP e interactúa con el cliente mediante estructuras de datos Pydantic (app/main.py):

```
from fastapi import FastAPI, UploadFile, File, HTTPException, status
from pydantic import BaseModel, Field
from app.rag import rag_core_orchestrator
import uuid
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("RAG_Production_API")

app = FastAPI(
    title="Motor RAG Empresarial API",
    version="1.0.0",
    description="Servicio corporativo de consulta documental inteligente mediante
embeddings locales y LLM Cloud."
)

class QueryInput(BaseModel):
    question: str = Field(..., min_length=5, description="Pregunta en lenguaje natural
redactada por el usuario", example="¿Cuál es la vigencia del contrato?")
    document_id: str = Field(..., description="Identificador UUID único del documento
previamente indexado")

class QueryOutput(BaseModel):
    answer: str = Field(..., description="Respuesta sintetizada e hilada por el modelo
de lenguaje")
    document_id: str = Field(..., description="ID del documento analizado")

@app.post("/upload", status_code=status.HTTP_201_CREATED, tags=["Administración
Documental"])
async def upload_document(file: UploadFile = File(...)):
    # Validación estricta perimetral de extensiones autorizadas
    if not file.filename.endswith(('.txt', '.pdf')):
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Archivo
inválido. Solo se admiten formatos (.txt) o (.pdf)")

    try:
        raw_bytes = await file.read()
        # Decodificación estándar de texto plano (para PDFs reales se integraría un
extractor como PyPDF2)
        extracted_text = raw_bytes.decode("utf-8", errors="ignore")
```

```
        generated_uuid = str(uuid.uuid4())
        total_chunks =
rag_core_orchestrator.process_document_ingestion(generated_uuid, extracted_text)

        logger.info(f"DOCUMENTO INDEXADO: ID={generated_uuid} | Chunks
creados={total_chunks}")
        return {
            "document_id": generated_uuid,
            "status": "indexed",
            "chunks_generated": total_chunks
        }
    except Exception as error:
        logger.error(f"FALLO CRÍTICO EN INGESTA: {str(error)}")
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
detail=f"Fallo interno en el pipeline de ingesta: {str(error)}")

@app.post("/ask", response_model=QueryOutput, status_code=status.HTTP_200_OK,
tags=["Conversación Semántica"])
def ask_document(payload: QueryInput):
    try:
        ai_response =
rag_core_orchestrator.resolve_knowledge_query(payload.document_id, payload.question)
        return QueryOutput(answer=ai_response, document_id=payload.document_id)
    except Exception as error:
        logger.error(f"FALLO EN CONSULTA RAG: {str(error)}")
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
detail=f"Error al procesar la inferencia semántica: {str(error)}")
```

9. Despliegue y Docker

El esquema de infraestructura para el entorno RAG requiere especial cuidado debido a la coexistencia de componentes estáticos locales (pesos del modelo de embeddings descargados en disco) y llamadas de red externas hacia el endpoint de la API de Google Gemini.

Se implementa un entorno contenerizado profesional que aísla de forma persistente el directorio de los modelos de sentence-transformers y expone los tokens secretos a través de inyecciones nativas de variables de entorno, evitando el almacenamiento de credenciales en las capas de código:

```
FROM python:3.10-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    XDG_CACHE_HOME=/app/models_cache  # Redirección de la caché del descargador de
HuggingFace

WORKDIR /app
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

COPY ./app ./app
RUN mkdir -p /app/models_cache /app/vectorstore

RUN useradd -u 8888 raguser && chown -R raguser:raguser /app
USER raguser

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10. Estructura del proyecto

La distribución física de componentes asegura un aislamiento total del Vector Store local y de los binarios lingüísticos, facilitando políticas de backup e inmutabilidad del software:

```
/
├── app/
│   ├── __init__.py      # Inicializador del módulo REST
│   ├── main.py          # Definición de enrutamiento HTTP y esquemas Pydantic
│   ├── rag.py           # Núcleo lógico del Pipeline RAG y Prompt Engineering
│   └── embeddings.py    # Motor local de vectores y algoritmos de similitud coseno
├── models_cache/        # Directorio persistente donde se guardan los pesos de all-
MiniLM-L6-v2
├── vectorstore/         # Base de datos vectorial persistente
├── Dockerfile           # Receta de la imagen de producción optimizada para IA
└── requirements.txt     # Manifiesto bloqueado (fastapi, uvicorn, sentence-
transformers, openai, numpy)
```

11. Buenas prácticas

El despliegue de soluciones basadas en LLMs comerciales expone a las organizaciones a riesgos financieros severos debido al consumo desmedido de tokens si no se aplican barreras defensivas de software:

- **Evitar prompts largos innecesarios y sobrecostes:** Diseñar ventanas de contexto ajustadas estrictamente a las evidencias recuperadas. Jamás debe enviarse el documento

completo al LLM en cada pregunta, inyectando únicamente los fragmentos con scores de similitud superiores a umbrales rigurosos.
- **Controlar y auditar el consumo de tokens:** Implementar validaciones e interceptores sintácticos que acoten y rechacen preguntas que excedan un número razonable de caracteres por parte del usuario, blindando el presupuesto operativo de la organización contra ataques destructivos de denegación de billetera (Wallet Denial of Service).
- **Cachear resultados de consultas frecuentes:** Implementar capas de almacenamiento en caché intermedia (como Redis) para almacenar pares de Pregunta-Respuesta idénticos. Si un usuario formula exactamente la misma pregunta sobre el mismo documento, el sistema responde desde la caché en microsegundos, eliminando a cero el coste de inferencia del LLM.

12. Escalabilidad

Para transicionar este prototipo local de alta fidelidad hacia un entorno con capacidad para dar servicio a millones de usuarios concurrentes analizando billones de documentos corporativos, se definen las siguientes líneas de evolución de arquitectura:

- **Migración hacia bases de datos vectoriales empresariales (Pinecone):** Sustituir el motor lineal de similitud en memoria de `app/embeddings.py` por una base de datos vectorial de nivel empresarial distribuida y gestionada en la nube, como Pinecone, Milvus o Qdrant, las cuales utilizan índices HNSW (Hierarchical Navigable Small World) capaces de resolver búsquedas espaciales en microsegundos sobre miles de millones de vectores de forma nativa.
- **Paralelización asíncrona y desacoplamiento de pipelines:** Separar físicamente el pipeline de ingesta (carga, limpieza y embedding del documento) del pipeline síncrono de consulta, convirtiendo la indexación en un proceso asíncrono gestionado por workers independientes en segundo plano.
- **Arquitectura distribuida basada en microservicios autónomos:** Dividir el monolito en microservicios independientes: un servicio dedicado exclusivamente a la generación rápida de embeddings numéricos (escalable mediante hardware GPU) y otro dedicado al control de flujos conversacionales REST.

13. Conclusión

El diseño y desarrollo completo del sistema RAG presentado consolida una solución madura y de alto valor estratégico para la gestión del conocimiento moderno dentro de las organizaciones. Al anclar las capacidades lingüísticas abstractas de los modelos generativos a una base vectorial de datos factuales inmutables, se dota a la empresa de una herramienta conversacional inteligente que es, al mismo tiempo, creativa en su redacción y estrictamente veraz en sus afirmaciones.

Este patrón arquitectónico se establece como la metodología estándar en la industria de la Inteligencia Artificial aplicada para interactuar de forma segura, económica y eficiente con

repositorios de conocimiento privados, demostrando una sólida madurez en el diseño de arquitecturas de software modernas de nivel de producción.